

AADK 8: Task 2 — Art Space App: Design, States, & Test Plan

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 8

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Feature & UI Design

Main Feature

The enhanced **Art Space App** allows users to **browse artworks, view artwork details, and save favourite artworks**. Users can navigate between artworks using **Next/Previous buttons**, open a **details view**, and mark artworks as **favourites**.

Main Screens

1. Gallery View

- Displays the artwork image
- Shows artwork title and artist
- Includes **Next** and **Previous** buttons
- Includes a **Favourite (Heart) icon**

Composable Components

- Image – Displays artwork
 - Text – Title and artist
 - Button – Next / Previous navigation
 - IconButton – Favourite toggle
-

2. Details View

- Shows larger artwork image
- Displays artwork title, artist, and year

- Includes a **Back button**

Composable Components

- Image
 - Text
 - Button
-

3. Favourites View

- Displays list of saved artworks
- Allows user to view details of favourite artwork

Composable Components

- LazyColumn
 - Image
 - Text
-

UI Flow

1. User opens **Gallery View**
 2. User presses **Next / Previous**
 3. Artwork updates
 4. User presses **Heart Icon**
 5. Artwork added to **Favourites**
 6. User opens **Details View**
 7. User returns to gallery
-

2. State Variables & Logic

State Variable	Type	Initial Value	Description	Transition
currentArtworkIndex	Int	0	Tracks the current artwork being displayed	Tap Next → currentArtworkIndex++
favourites	List	Empty list	Stores artworks marked as favourites	Tap Heart Icon → add/remove artwork
showDetails	Boolean	false	Controls whether details screen is visible	Tap artwork → showDetails = true
artworks	List	Predefined list	List of all artworks	Static data source

Example Logic

- Tap **Next Button** → Increase artwork index and display next artwork.
- Tap **Previous Button** → Decrease artwork index.
- Tap **Heart Icon** → Toggle favourite state.
- Tap **Artwork Image** → Open details screen.

3. Components & Testable Units

Unit 1 — Next Button Navigation

UI Components

- Button
- Image
- Text

Trigger Action

User taps **Next button**

Expected Result

- currentArtworkIndex increases

- Image updates
 - Artwork title changes
-

Unit 2 — Favourite Icon Toggle

UI Components

- IconButton (Heart icon)

Trigger Action

User taps the **Favourite icon**

Expected Result

- Artwork added to favourites list
 - Icon state changes (filled heart)
-

Unit 3 — Details Screen Data Display

UI Components

- Image
- Text (title, artist, year)

Trigger Action

User taps artwork image

Expected Result

- Details screen opens
 - Correct artwork data is displayed
-

4. Test Plan Table

Test ID	Unit Under Test	Test Type	Preconditions	Trigger	Expected Outcome
T01	Next Button Navigation	Instrumented UI Test	App launched with artwork index = 0	Tap Next button	Next artwork image and title displayed

Test ID	Unit Under Test	Test Type	Preconditions	Trigger	Expected Outcome
T02	Previous Button Navigation	Instrumented UI Test	Artwork index > 0	Tap Previous button	Previous artwork displayed
T03	Favourite Toggle	Unit + UI Test	Artwork visible on gallery screen	Tap Heart icon	Artwork added to favourites list
T04	Details Screen Display	Instrumented UI Test	Gallery screen loaded	Tap artwork image	Details screen shows correct artwork info
T05	Edge Case – End of List	Unit Test	Current artwork is last in list	Tap Next button	Index resets or stays at last artwork
T06	Edge Case – Duplicate Favourite	Unit Test	Artwork already in favourites	Tap Heart icon again	Artwork removed from favourites
T07	Orientation Change	Instrumented UI Test	Artwork index = 2	Rotate device	Same artwork remains displayed

5. Reflection

Planning tests together with the design helps improve application reliability because it ensures that every feature has clear expected behavior before implementation. Testing helps detect errors early and ensures UI components respond correctly to user interactions. One difficulty when implementing tests is that UI tests can be slower and sometimes unstable compared to business logic tests. Business logic tests are easier because they do not depend on the UI environment. To reduce flakiness in UI tests, stable selectors such as test tags and controlled state management will be used.